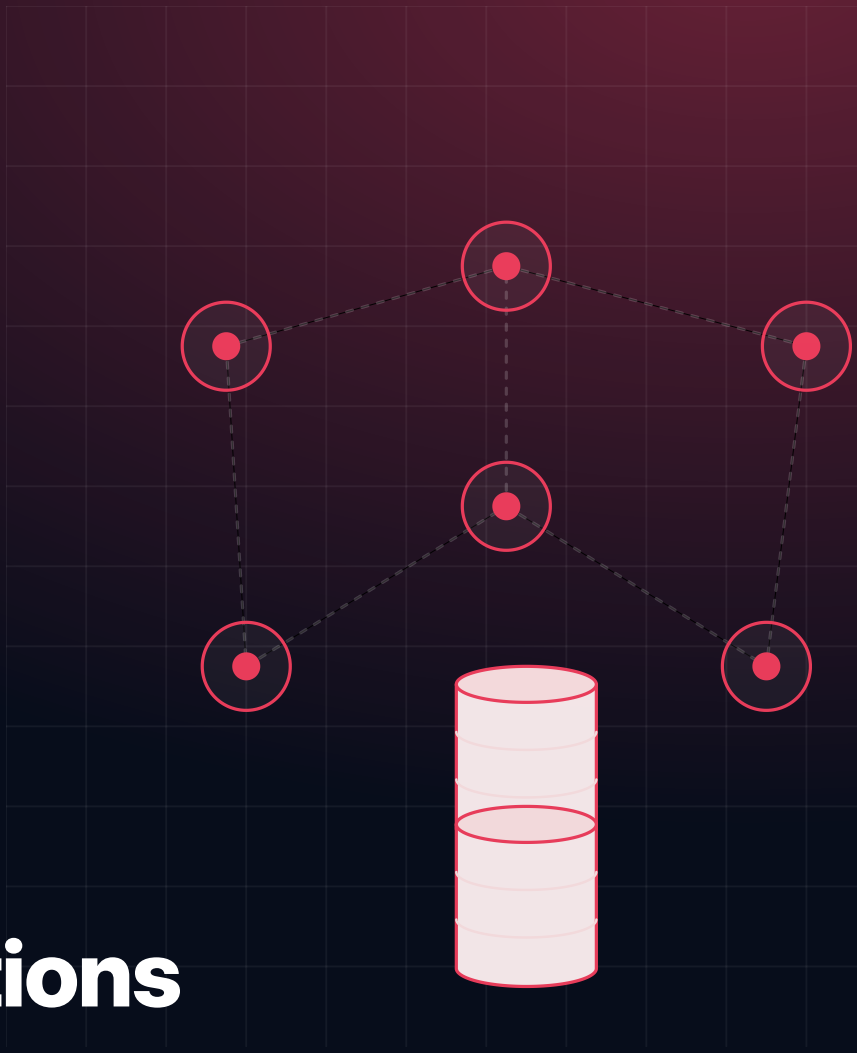


PHASE 05 / 07

Functions & Transformations

The toolbox for cleaning and reshaping values: numeric, string and date functions, type casting, NULL handling and CASE logic.

Numeric Functions | String Functions | Date Functions | Casting | CASE & NULLs

Raw data is never clean. This phase gives you the functions to standardize it, and shows how real warehouses use them to turn staging tables into trustworthy ones.

CONTENTS

Inside this phase

Numeric Functions & Arithmetic 3

- Three families of functions 3
- Arithmetic on columns 4
- Rounding: four functions, four answers 5
- A derived metric: discount percentage 6

String Functions 7

- The core toolkit 7
- Seeing positions: how cutting functions work 9
- Real cleaning: the messy phone column 10

Date & Time Functions 12

- Now: current date and time 12
- Pulling a date apart 13
- Date arithmetic 14
- Formatting and parsing dates 15
- Date filters that stay fast (SARGability) 16

Type Casting 17

- CAST and CONVERT 17
- The implicit-conversion trap 18

CASE Logic & NULL Handling 19

- CASE WHEN: first match wins 19
- Conditional aggregation: the pivot trick 21
- Handling NULLs 22

Putting It Together: A Cleaning Layer 24

Phase 5 Recap & Practice 26

- Cheat sheet 26
- Check yourself 26
- Practice lab 27
- Answer key 27

5.1 PART 5.1 · TRANSFORM: NUMBERS

Numeric Functions & Arithmetic

So far SELECT has *returned* columns. Now it will *compute* new ones. A **function** takes values in and gives a value back; combined with arithmetic operators, they let you derive prices, percentages and metrics inside the query itself.

What you will learn

- Tell scalar, aggregate and window functions apart
- Calculate with `+ − × ÷`, `DIV`, `MOD` and understand NULL and divide-by-zero behavior
- Choose between `ROUND`, `FLOOR`, `CEIL` and `TRUNCATE`
- Build derived metrics such as discount percentage

Why it matters for data engineering. Transformation is the "T" in ETL/ELT. Almost every metric in a warehouse (margin, growth, conversion rate) is a derived column built from these functions, and rounding mistakes in finance numbers are expensive.

Three families of functions

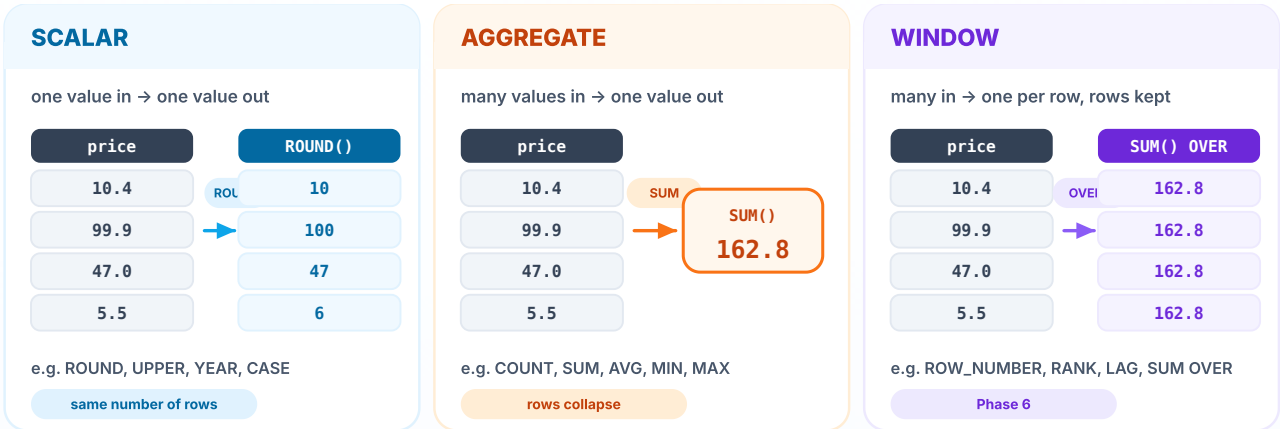


Figure 1. This phase is about **scalar** functions (blue). You already met aggregates in Phase 3; window functions arrive in Phase 6.

Arithmetic on columns

Any column can take part in an expression. The course uses a 10 % discount, a flat addition, division and a square:

SQL — Course ch-6 'Numeric Transformations'

```sql
SELECT product_name,
       unit_price,
       unit_price * 0.90        AS discounted_price,
       unit_price + 10          AS taxed_price,
       unit_price / 10          AS fractioned_price,
       ROUND(unit_price, 1)     AS rounded_price,
       unit_price * unit_price  AS multiply_price
FROM   dim_product
ORDER  BY product_key
LIMIT  4;
```

RESULT — 4 ROWS

product_name	unit_price	discounted_price	taxed_price	fractioned_price	rounded_price	multiply_price
Figure Method	897.35	807.6150	907.35	89.735000	897.4	805237.0225
Pressure That	920.58	828.5220	930.58	92.058000	920.6	847467.5364
Huge Change	553.56	498.2040	563.56	55.356000	553.6	306428.6736
Plant Agency	97.33	87.5970	107.33	9.733000	97.3	9473.1289

Operator	Meaning	Example → result
`+` `-` `*`	add, subtract, multiply	`2 + 3 * 4` → 14 (× before +)
`/`	true division (decimals)	`7 / 2` → 3.5000
`DIV`	integer division, drops the remainder	`7 DIV 2` → 3
`%` or `MOD(a,b)`	remainder	`7 % 2` → 1 (handy for even/odd)
`POWER(a,b)` · `SQRT(x)` · `ABS(x)`	power, square root, absolute value	`POWER(2,10)` → 1024
`GREATEST(a,b…)` · `LEAST(a,b…)`	largest / smallest of several values (per row)	`LEAST(5, 9, 2)` → 2

⚠ WATCH OUT

NULL is contagious: `5 + NULL` is NULL, and so is `NULL * 0`. And in MySQL `10 / 0` does not crash a SELECT; it quietly returns **NULL** (with a warning). In INSERT/UPDATE under strict mode it raises an error. Guard denominators: `a / NULLIF(b, 0)`.

SQL — NULL propagation and division behavior

```sql
SELECT 5 + NULL AS add_null, 10 / 0 AS div_zero, 10 / NULLIF(0, 0) AS safe_div, 7 / 2 AS true_div, 7 DIV 2 AS int_div, 7 % 2 AS remainder;
```

RESULT — 1 ROW

add_null	div_zero	safe_div	true_div	int_div	remainder
NULL	*NULL*	*NULL*	3.5000	3	1

Rounding: four functions, four answers

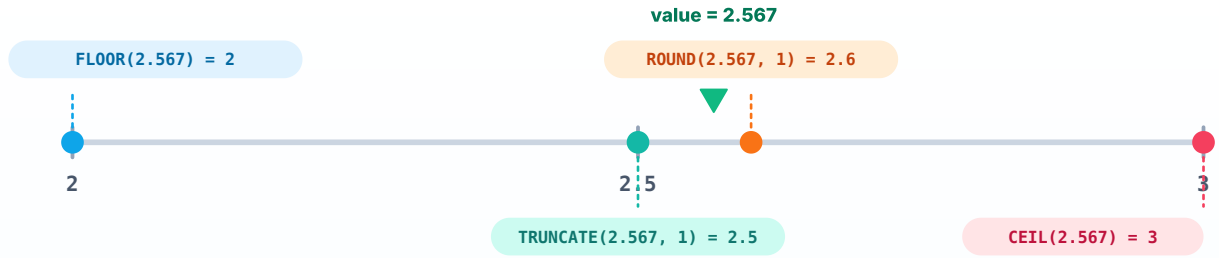


Figure 2. Where each rounding function lands for 2.567. TRUNCATE chops; ROUND rounds to nearest; FLOOR/CEIL always move down/up to a whole number.

SQL — Negative numbers: FLOOR(-2.1) = -3 · ROUND rounds halves away from zero for exact DECIMALs

```sql
SELECT ROUND(2.567, 1) AS round_1dp,  ROUND(2.5) AS round_2_5,  ROUND(-2.5) AS round_neg,
       TRUNCATE(2.567, 1) AS trunc,    FLOOR(-2.1) AS floor_neg, CEIL(2.1) AS ceil_pos;
```

RESULT — 1 ROW

round_1dp	round_2_5	round_neg	trunc	floor_neg	ceil_pos
2.6	3	-3	2.5	-3	3

MYSQL SPECIFIC

`ROUND(2.5)` gives 3 for exact values (`DECIMAL` , `INT`): halves round **away from zero**. For approximate `FLOAT` / `DOUBLE` values MySQL uses the C library, which on most platforms rounds an exact half to the **nearest even** number (2.5 → 2). Another reason to store money as `DECIMAL` .

A derived metric: discount percentage

Net total in `fact_sales` is `quantity_sold × unit_price − discount`. Let's derive the *discount rate* as a percentage of the gross value, a typical business metric. Rounding to 1 decimal keeps it readable.

SQL — gross → discount → net → percentage

```sql
SELECT sales_id,
       quantity_sold * unit_price                                AS gross,
       discount,
       total_amount                                              AS net,
       ROUND(100 * discount / (quantity_sold * unit_price), 1)   AS discount_pct
FROM   fact_sales
ORDER  BY sales_id
LIMIT  5;
```

RESULT — 5 ROWS

sales_id	gross	discount	net	discount_pct
1	129.68	35.64	94.04	27.5
2	1362.48	37.93	1324.55	2.8
3	1037.92	33.39	1004.53	3.2
4	5991.66	42.55	5949.11	0.7
5	398.25	38.12	360.13	9.6

SQL — The six negative sales from Phase 3: the discount exceeds 100 % of the gross value

```sql
SELECT   sales_id, quantity_sold * unit_price AS gross, discount, total_amount AS net,
         ROUND(100 * discount / (quantity_sold * unit_price), 1) AS discount_pct
FROM     fact_sales
WHERE    total_amount < 0
ORDER BY discount_pct DESC
LIMIT    3;
```

RESULT — 3 ROWS

sales_id	gross	discount	net	discount_pct
3300	15.91	43.38	-27.47	272.7
414	10.64	24.16	-13.52	227.1
4331	23.92	49.68	-25.76	207.7

DATA-ENGINEERING LENS

A discount above 100 % is a business impossibility. A derived metric made the problem *visible*: this is exactly how transformation layers double as data-quality checks. Cap, flag or quarantine, but never ignore.

5.2 PART 5.2 · TRANSFORM: TEXT

String Functions

Text is the messiest data type: inconsistent case, stray spaces, mixed formats, packed values. String functions are your cleaning tools.

What you will learn

- Join, measure, change case and trim text
- Cut text with `SUBSTRING`, `LEFT`, `RIGHT`, `SUBSTRING_INDEX`
- Search and replace with `INSTR`, `REPLACE`, `REGEXP_REPLACE`
- Standardize a real column (phone numbers)

Why it matters for data engineering. Joins fail silently when keys differ by case or whitespace (`'abc '` ≠ `'ABC'`). Standardizing text in the staging layer is the cheapest way to prevent mismatches, duplicates and broken dashboards downstream.

The core toolkit

SQL — Course ch-6 'String Functions' (1/2): CONCAT, CONCAT_WS, LENGTH

```sql
SELECT first_name, last_name, country,
       CONCAT(first_name, ' ', last_name)           AS full_name,
       CONCAT_WS(' ', first_name, last_name, country) AS full_with_country,
       LENGTH(country)                              AS country_size
FROM   dim_customer
ORDER  BY customer_key
LIMIT  3;
```

RESULT — 3 ROWS

first_name	last_name	country	full_name	full_with_country	country_size
Antonio	Kelly	Tunisia	Antonio Kelly	Antonio Kelly Tunisia	7
Todd	Lucas	Djibouti	Todd Lucas	Todd Lucas Djibouti	8
Matthew	Lee	Guinea-Bissau	Matthew Lee	Matthew Lee Guinea-Bissau	13

SQL — Course ch-6 'String Functions' (2/2)

```sql
SELECT email,
       LOWER(city)               AS city_lower,
       SUBSTRING(email, 1, 6)    AS first_6,
       REPLACE(email, '@', '%')  AS replaced,
       LEFT(country, 3)          AS left_3,
       RIGHT(country, 3)         AS right_3,
       REVERSE(country)          AS reversed,
       REPEAT(first_name, 2)     AS repeated
FROM   dim_customer
ORDER  BY customer_key
LIMIT  3;
```

RESULT — 3 ROWS

email	city_lower	first_6	replaced	left_3	right_3	reversed	repeated
richard16@example.net	lake timothy	richar	richard16%example.net	Tun	sia	aisinuT	AntonioAntonio
danielsnyder@example.…	nicholasstad	daniel	danielsnyder%example.…	Dji	uti	ituobijD	ToddTodd
joseph40@example.net	janiceton	joseph	joseph40%example.net	Gui	sau	uassiB-aeniuG	MatthewMatthew

Function	Does	Example → result
`CONCAT(a,b,…)`	join text (NULL anywhere → NULL!)	`CONCAT('a','b')` → ab
`CONCAT_WS(sep,a,b,…)`	join with a separator, **skips NULLs**	`CONCAT_WS('-','x',NULL,'z')` → x-z
`LENGTH` · `CHAR_LENGTH`	bytes · characters	`CHAR_LENGTH('café')` → 4, `LENGTH` → 5
`UPPER` · `LOWER`	change case	`UPPER('Sql')` → SQL
`TRIM` · `LTRIM` · `RTRIM`	remove spaces (or a given character)	`TRIM(' hi ')` → hi
`LPAD(s,n,c)` · `RPAD`	pad to a fixed length	`LPAD('7',3,'0')` → 007
`SUBSTRING(s,pos,len)`	cut from position (1-based!)	`SUBSTRING('abcdef',2,3)` → bcd
`LEFT(s,n)` · `RIGHT(s,n)`	first / last n characters	`RIGHT('abcdef',2)` → ef
`INSTR(s,x)` · `LOCATE(x,s)`	position of text (0 = not found)	`INSTR('a@b','@')` → 2
`REPLACE(s,from,to)`	replace every occurrence	`REPLACE('a-b','-','')` → ab
`SUBSTRING_INDEX(s,d,n)`	part before/after n-th delimiter	see below
`REGEXP_REPLACE(s,pat,rep)`	regex search and replace (MySQL 8)	strip non-digits

Seeing positions: how cutting functions work

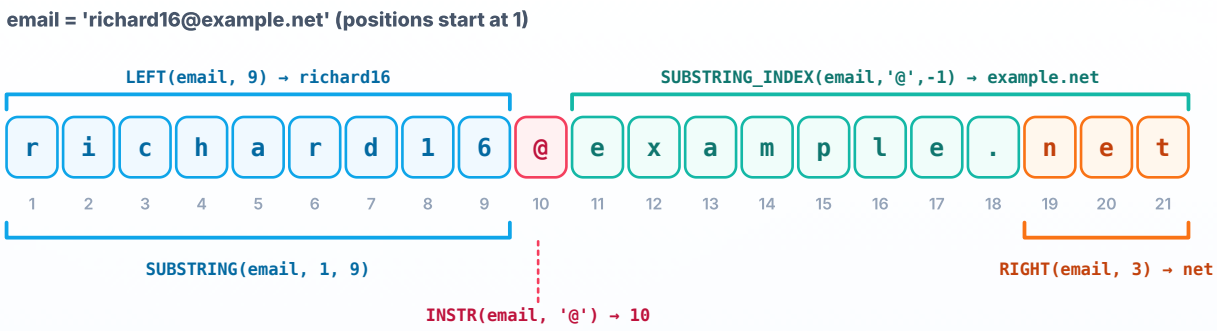


Figure 3. Index every character (starting at 1) and the cutting functions become simple counting.

SQL — Split an email into username and domain: a common cleaning step

```sql
SELECT email,
       INSTR(email, '@')                      AS at_position,
       SUBSTRING(email, 1, INSTR(email,'@')-1) AS username,
       SUBSTRING_INDEX(email, '@', -1)         AS domain
FROM   dim_customer
ORDER  BY customer_key
LIMIT  3;
```

RESULT — 3 ROWS

email	at_position	username	domain
richard16@example.net	10	richard16	example.net
danielsnyder@example.org	13	danielsnyder	example.org
joseph40@example.net	9	joseph40	example.net

SQL — ...and analyze it: customers per email domain

```sql
SELECT   SUBSTRING_INDEX(email, '@', -1) AS domain, COUNT(*) AS customers
FROM     dim_customer
GROUP BY domain
ORDER BY customers DESC;
```

RESULT — 3 ROWS

domain	customers
example.net	179
example.org	174
example.com	147

Real cleaning: the messy phone column

The `phone` column in `dim_customer` mixes lengths and prefixes. A quick profile with `LENGTH` shows the problem:

SQL — Phone-number lengths in the raw data

```sql
SELECT LENGTH(phone) AS digits, COUNT(*) AS customers FROM dim_customer GROUP BY LENGTH(phone) ORDER BY digits;
```

RESULT — 5 ROWS

digits	customers
10	163
11	16
13	74
14	78
15	169

SQL — Keep the last 10 digits as the national number; the rest is a country/dial prefix

```sql
SELECT customer_key,
       phone                                AS raw_phone,
       RIGHT(phone, 10)                     AS last_10_digits,
       LEFT(phone, LENGTH(phone) - 10)      AS prefix
FROM   dim_customer
ORDER  BY customer_key
LIMIT  4;
```

RESULT — 4 ROWS

customer_key	raw_phone	last_10_digits	prefix
1	9904005277	9904005277	
2	0012907800627	2907800627	001
3	001596505299143	6505299143	00159
4	182656936555456	6936555456	18265

SQL — Regex strips every non-digit (useful when phones hold spaces, brackets, dashes)

```sql
SELECT REGEXP_REPLACE('(+91) 98765-43210', '[^0-9]', '') AS digits_only;
```

RESULT — 1 ROW

digits_only
919876543210

DATA-ENGINEERING LENS

Clean in layers. Keep the raw value (`raw_phone`) untouched, and add the cleaned value (`phone_clean`) alongside. If your rule turns out to be wrong you can re-derive it; if you overwrote the raw data you cannot.

⚠ WATCH OUT

`CONCAT('a', NULL)` is **NULL**, so a missing middle name wipes out the whole full name. Use `CONCAT_WS(' ', first, middle, last)` (skips NULLs) or wrap with `COALESCE`.

5.3 PART 5.3 · TRANSFORM: TIME

Date & Time Functions

Almost every business question has a time dimension: *per month*, *since signup*, *week over week*. MySQL's date functions pull parts out of dates, do date arithmetic and format dates for display.

What you will learn

- Get the current date/time and know UTC vs local
- Extract year, month, day, weekday and quarter
- Add, subtract and measure differences between dates
- Format and parse dates with `DATE_FORMAT` and `STR_TO_DATE`
- Write date filters that can use an index

Why it matters for data engineering. Time bucketing (daily, monthly) is the backbone of fact tables and reports. Off-by-one-day errors from time zones, or a function wrapped around a date column that kills index use, are two of the most common real-world performance and accuracy bugs.

Now: current date and time

SQL — Course ch-6 'DATE Transformations' (1): your output will show the current moment

```sql
SELECT NOW() AS now_local, CURDATE() AS today, UTC_DATE() AS date_utc, UTC_TIME() AS time_utc,
UTC_TIMESTAMP() AS ts_utc;
```

RESULT — 1 ROW

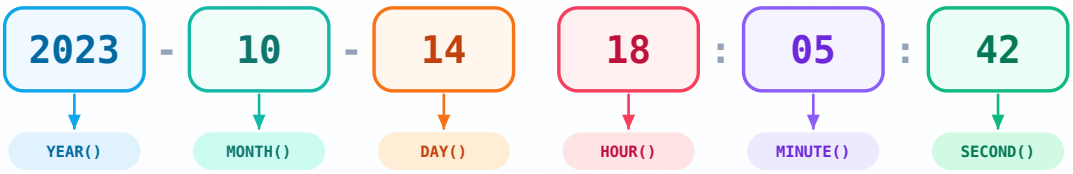
now_local	today	date_utc	time_utc	ts_utc
2026-10-17 22:59:25	2026-10-17	2026-10-17	22:59:25	2026-10-17 22:59:25

DATA-ENGINEERING LENS

Store event times in **UTC** and convert only for display. `NOW()` follows the server/session time zone; `UTC_TIMESTAMP()` does not. Mixing the two is a classic source of “my daily numbers are shifted by one day”.

Pulling a date apart

A DATETIME value: 2023-10-14 18:05:42



Also: DAYNAME() → Saturday · WEEKDAY() → 5 (Mon = 0) · DAYOFWEEK() → 7 (Sun = 1) · QUARTER() → 4 · MONTHNAME() → October
DATE(x) keeps only the date part, TIME(x) only the time part.

Figure 4. One timestamp, many parts. Each function returns a number or a name.

SQL — Course ch-6 'DATE Transformations' (2): parts of a date

```sql
SELECT date,
       YEAR(date)      AS yr,
       MONTH(date)     AS mth,
       DAY(date)       AS dy,
       WEEKDAY(date)   AS weekday_no,
       DAYNAME(date)   AS day_name,
       QUARTER(date)   AS qtr
FROM   dim_date
ORDER  BY date
LIMIT  3;
```

RESULT — 3 ROWS

date	yr	mth	dy	weekday_no	day_name	qtr
2023-10-14	2023	10	14	5	Saturday	4
2023-10-15	2023	10	15	6	Sunday	4
2023-10-16	2023	10	16	0	Monday	4

NOTE

Your `dim_date` table stores these parts already (`month_name` , `quarter` , `is_weekend`). **That is the point of a date dimension:** compute calendar attributes once, then join, instead of calling functions on millions of fact rows.

Date arithmetic

SQL — Course ch-6: DATEDIFF, ADDDATE, SUBDATE · plus DATE_ADD and LAST_DAY

```sql
SELECT date,
       DATEDIFF('2025-12-31', date)        AS days_until_end_2025,
       ADDDATE(date, 2)                    AS plus_2_days,
       SUBDATE(date, 2)                    AS minus_2_days,
       DATE_ADD(date, INTERVAL 1 MONTH)    AS plus_1_month,
       LAST_DAY(date)                      AS month_end
FROM   dim_date
ORDER  BY date
LIMIT  3;
```

RESULT — 3 ROWS

date	days_until_end_2025	plus_2_days	minus_2_days	plus_1_month	month_end
2023-10-14	809	2023-10-16	2023-10-12	2023-11-14	2023-10-31
2023-10-15	808	2023-10-17	2023-10-13	2023-11-15	2023-10-31
2023-10-16	807	2023-10-18	2023-10-14	2023-11-16	2023-10-31

Goal	Function	Example → result
Days between two dates	`DATEDIFF(later, earlier)`	`DATEDIFF('2025-12-31','2025-12-01')` → 30
Difference in any unit	`TIMESTAMPDIFF(unit, start, end)`	`TIMESTAMPDIFF(MONTH,'2024-01-31','2024-03-30')` → 1
Add / subtract a period	`DATE_ADD` / `DATE_SUB` + `INTERVAL`	`DATE_ADD('2024-01-31', INTERVAL 1 MONTH)` → 2024-02-29
Last day of month	`LAST_DAY(d)`	`LAST_DAY('2024-02-10')` → 2024-02-29
Extract a part	`EXTRACT(unit FROM d)`	`EXTRACT(YEAR_MONTH FROM '2025-03-14')` → 202503

SQL — How much customer history do we hold?

```sql
SELECT   TIMESTAMPDIFF(YEAR, MIN(join_date), '2025-12-31') AS years_of_history,
         DATEDIFF('2025-12-31', MIN(join_date))            AS days_of_history,
         MIN(join_date) AS first_signup
FROM     dim_customer;
```

RESULT — 1 ROW

years_of_history	days_of_history	first_signup
5	1906	2020-10-12

Formatting and parsing dates

`DATE_FORMAT(date, pattern)` turns a date into text; `STR_TO_DATE(text, pattern)` does the reverse. They share the same pattern codes.

Code	Meaning	Code	Meaning
`%Y`	4-digit year (2025)	`%W`	weekday name (Friday)
`%y`	2-digit year (25)	`%a`	short weekday (Fri)
`%m`	month 01–12	`%H`	hour 00–23
`%M`	month name (March)	`%h`	hour 01–12
`%b`	short month (Mar)	`%i`	minutes 00–59
`%d`	day 01–31	`%s`	seconds 00–59
`%e`	day 1–31 (no zero)	`%p`	AM / PM

SQL — Course ch-6 'DATE Transformations' (3) · plus the YYYY-MM month bucket

```sql
SELECT date, DATE_FORMAT(date, '%W %M %e %Y') AS converted_date, DATE_FORMAT(date, '%Y-%m') AS ym
FROM dim_date ORDER BY date LIMIT 3;
```

RESULT — 3 ROWS

date	converted_date	ym
2023-10-14	Saturday October 14 2023	2023-10
2023-10-15	Sunday October 15 2023	2023-10
2023-10-16	Monday October 16 2023	2023-10

SQL — Parse European and long-form text dates into real DATEs

```sql
SELECT STR_TO_DATE('14/03/2025', '%d/%m/%Y') AS parsed_date, STR_TO_DATE('March 5, 2024', '%M %e, %Y') AS parsed_text;
```

RESULT — 1 ROW

parsed_date	parsed_text
2025-03-14	2024-03-05

Date filters that stay fast (SARGability)

A condition is **SARGable** (*Search ARGument-able*) when the database can use an index to jump straight to matching rows. Wrapping the *column* in a function usually breaks that: the engine must compute the function for **every row**.

× Function on the column

```
WHERE YEAR(date) = 2024
```

computes YEAR() for every row,
then compares: full table scan

✓ Range on the raw column

```
WHERE date >= '2024-01-01'
 AND date < '2025-01-01'
```

index range seek: jumps to Jan 1
and stops at Dec 31

Figure 5. Same answer, very different cost. The rule: leave the column alone, transform the constants.

SQL — Identical results (366 days in leap-year 2024): only the performance differs

```sql
SELECT (SELECT COUNT(*) FROM dim_date WHERE YEAR(date) = 2024)                    AS
with_function,
       (SELECT COUNT(*) FROM dim_date WHERE date >= '2024-01-01' AND date < '2025-01-01')    AS
with_range;
```

RESULT — 1 ROW

with_function	with_range
366	366

INTERVIEW ANGLE

"Why is `WHERE DATE(created_at) = '2025-03-14'` slow on a big table?" The function on the column prevents index use. Rewrite: `created_at >= '2025-03-14' AND created_at < '2025-03-15'`.

5.4 PART 5.4 · CHANGING DATA TYPES

Type Casting

Casting converts a value from one data type to another. MySQL sometimes does it for you silently (*implicit*), which is convenient and dangerous. Explicit casting with `CAST()` says exactly what you mean.

What you will learn

- Convert values with `CAST()` and `CONVERT()`
- Know which casts are safe and which lose information
- Spot the string-vs-number comparison trap
- Cast text into dates and decimals when loading raw files

Why it matters for data engineering. Raw files (CSV, JSON, API dumps) arrive as text. The first transformation in most pipelines is casting each column to its proper type, and a wrong cast (or a missing one) is a classic cause of silent data corruption.

CAST and CONVERT



Casting targets in MySQL: CHAR · SIGNED / UNSIGNED (integers) · DECIMAL(p,s) · DATE · DATETIME · TIME · JSON · FLOAT/DOUBLE (8.0.17+)
There is no CAST AS INT or VARCHAR in MySQL: use SIGNED and CHAR.

Figure 6. The three type families and the casts between them.

SQL — Course ch-6 'TYPE CASTING' plus other common casts

```sql
SELECT customer_key,
       CAST(customer_key AS CHAR(100))     AS key_as_text,
       CAST('2025-01-01' AS DATETIME)      AS text_to_datetime,
       CAST('42.9' AS SIGNED)              AS text_to_int,
       CAST('42.9' AS DECIMAL(5,1))        AS text_to_decimal,
       CONVERT('2025-06-30', DATE)         AS convert_to_date
FROM   dim_customer
ORDER  BY customer_key
LIMIT  2;
```

RESULT — 2 ROWS

customer_key	key_as_text	text_to_datetime	text_to_int	text_to_decimal	convert_to_date
1	1	2025-01-01 00:00:00	42	42.9	2025-06-30
2	2	2025-01-01 00:00:00	42	42.9	2025-06-30

⚠ WATCH OUT

`CAST('42.9' AS SIGNED)` truncates to **42**, no rounding and (without strict handling) only a warning. Casting `'abc'` to a number gives 0. Always profile for bad values before casting: `WHERE col NOT REGEXP '^[0-9]+$'`.

The implicit-conversion trap

Compare `'10'` and `'9'` as **text** and MySQL sorts character by character: `'1'` comes before `'9'`, so `'10' < '9'` is TRUE. Cast first and the comparison behaves.

SQL — 1 means TRUE, 0 means FALSE

```sql
SELECT '10' < '9' AS as_text, 10 < 9 AS as_numbers, CAST('10' AS UNSIGNED) < CAST('9' AS UNSIGNED)
AS cast_first;
```

RESULT — 1 ROW

as_text	as_numbers	cast_first
1	0	0

Situation	What MySQL does	Better practice
`'5' + 3`	converts text to number silently → 8	`CAST('5' AS SIGNED) + 3`
`WHERE varchar_id = 123`	converts **every** row's text to a number: index unusable	`WHERE varchar_id = '123'`
Joining `INT` to `VARCHAR`	implicit casts, possible full scans	cast one side explicitly in a staging step
Comparing `DATE` to `'2025-01-01'`	text converted to a date: fine	`DATE '2025-01-01'` or `CAST(… AS DATE)` for clarity

5.5 PART 5.5 · CONDITIONAL TRANSFORMATIONS

CASE Logic & NULL Handling

`CASE` is SQL's if/else. Together with the NULL-handling functions it handles bucketing, labelling, defaults and pivoting, the bread and butter of transformation layers.

What you will learn

- Write searched and simple `CASE` expressions
- Use `IF`, `IFNULL`, `COALESCE` and `NULLIF`
- Create buckets and flags from numeric columns
- Pivot rows into columns with conditional aggregation

Why it matters for data engineering. Business rules ("premium customer", "late order", "active user") live in CASE expressions. Conditional aggregation (`SUM(CASE …)`) is the standard way to build wide report tables without a PIVOT command.

CASE WHEN: first match wins

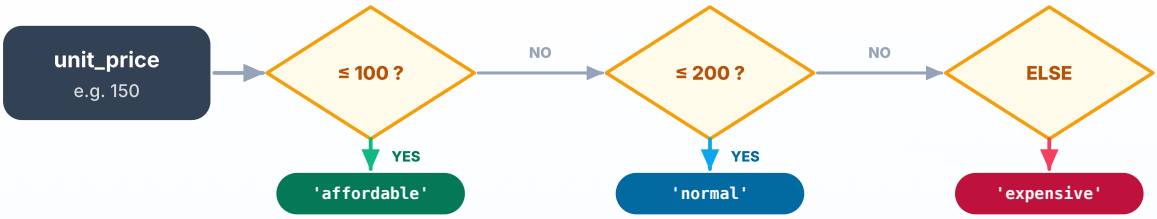


Figure 7. WHEN branches are tested top to bottom; the first TRUE branch wins and the rest are skipped. ELSE catches everything else (NULL if you omit it).

SQL — Course ch-7 CASE (1)

```sql
SELECT product_name, unit_price,
       CASE
           WHEN unit_price <= 100 THEN 'affordable'
           WHEN unit_price <= 200 THEN 'normal'
           ELSE 'expensive (but not for you!)'
       END AS price_category
FROM   dim_product
ORDER  BY product_key
LIMIT  4;
```

RESULT — 4 ROWS

product_name	unit_price	price_category
Figure Method	897.35	expensive (but not for you!)
Pressure That	920.58	expensive (but not for you!)
Huge Change	553.56	expensive (but not for you!)
Plant Agency	97.33	affordable

SQL — Course ch-7 CASE (2): conditions can combine several columns

```sql
SELECT product_name, category, unit_price,
       CASE
           WHEN unit_price <= 100 AND category = 'Clothing' THEN 'affordable'
           WHEN unit_price <= 200 AND category = 'Clothing' THEN 'normal'
           WHEN unit_price >  200 AND category = 'Clothing' THEN 'expensive (but not for you!)'
           ELSE CONCAT('Not for ', category)
       END AS price_category
FROM   dim_product
ORDER  BY product_key
LIMIT  5;
```

RESULT — 5 ROWS

product_name	category	unit_price	price_category
Figure Method	Clothing	897.35	expensive (but not for you!)
Pressure That	Clothing	920.58	expensive (but not for you!)
Huge Change	Clothing	553.56	expensive (but not for you!)
Plant Agency	Home & Kitchen	97.33	Not for Home & Kitchen
Fast Real	Home & Kitchen	333.97	Not for Home & Kitchen

SQL — CASE + GROUP BY: how are the 500 products distributed?

```sql
SELECT   CASE WHEN unit_price <= 100 THEN 'affordable'
              WHEN unit_price <= 200 THEN 'normal'
              ELSE 'expensive' END AS price_category,
         COUNT(*) AS products
FROM     dim_product
GROUP BY price_category
ORDER BY products DESC;
```

RESULT — 3 ROWS

price_category	products
expensive	402
affordable	49
normal	49

REMEMBER

Order your WHENs from most specific to most general. With `<= 100` first, a price of 50 never reaches the later branches. A **simple CASE** is the short form for equality: `CASE gender WHEN 'M' THEN 'Male' WHEN 'F' THEN 'Female' ELSE 'Unknown' END`.

Conditional aggregation: the pivot trick

Put a CASE *inside* an aggregate to count or sum only the rows that meet a condition. One query then returns a wide, report-style table: each category as a row, each quarter as a column.

SQL — Revenue by category (rows) × quarter (columns), a classic pivot

```sql
SELECT   p.category,
         ROUND(SUM(CASE WHEN d.quarter = 1 THEN f.total_amount END)) AS q1,
         ROUND(SUM(CASE WHEN d.quarter = 2 THEN f.total_amount END)) AS q2,
         ROUND(SUM(CASE WHEN d.quarter = 3 THEN f.total_amount END)) AS q3,
         ROUND(SUM(CASE WHEN d.quarter = 4 THEN f.total_amount END)) AS q4
FROM     fact_sales  f
JOIN     dim_date    d ON d.date_key    = f.date_key
JOIN     dim_product p ON p.product_key = f.product_key
GROUP BY p.category
ORDER BY p.category;
```

RESULT — 6 ROWS

category	q1	q2	q3	q4
Books	530587	642344	592142	585696
Clothing	660704	551207	540746	692203
Electronics	476416	559954	511332	484217
Home & Kitchen	606364	662116	614957	591178
Sports	603865	636491	651104	619447
Toys	595455	500407	506015	542508

DATA-ENGINEERING LENS

MySQL has no `PIVOT` keyword (SQL Server and Snowflake do). `SUM(CASE WHEN …)` works everywhere, and is how dbt pivot macros generate SQL. For a *dynamic* list of columns, build the statement text with a stored procedure or generate it in Python.

Handling NULLs

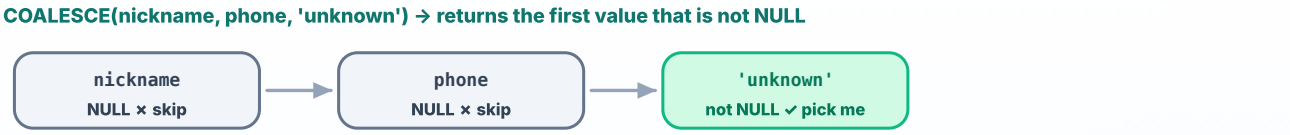


Figure 8. COALESCE walks left to right and stops at the first non-NULL value.

SQL — Four NULL tools on one table

```sql
SELECT id, nickname, phone, score,
       COALESCE(nickname, phone, 'unknown')  AS contact,
       IFNULL(score, 0)                      AS score_or_zero,
       IF(score IS NULL, 'missing', 'ok')    AS score_flag,
       NULLIF(score, 30)                     AS hide_30
FROM   nulls_demo
ORDER  BY id;
```

RESULT — 3 ROWS

id	nickname	phone	score	contact	score_or_zero	score_flag	hide_30
1	ace	NULL	10	ace	10	ok	10
2	NULL	555	NULL	555	0	missing	NULL
3	NULL	NULL	30	unknown	30	ok	NULL

Function	Meaning
`COALESCE(a, b, c …)`	first non-NULL argument (standard SQL; works everywhere)
`IFNULL(a, b)`	MySQL shortcut for two arguments
`NULLIF(a, b)`	returns NULL **if a = b**, else a: turns a bad sentinel (`0` , `''` , `'N/A'`) into a real NULL, or avoids ÷ 0
`IF(cond, x, y)`	MySQL's one-line if/else (a CASE with one WHEN)

SQL — COALESCE after a LEFT JOIN: replace the NULLs from unmatched rows

```sql
SELECT o.o_id, o.price, COALESCE(c.name, 'unknown customer') AS customer
FROM   orders o LEFT JOIN customers c ON o.cust_id = c.id
ORDER  BY o.o_id;
```

RESULT — 3 ROWS

o_id	price	customer
1	1000	love
2	1100	ansh
3	1200	unknown customer

5.6 PART 5.6 · APPLIED TRANSFORMATION

Putting It Together: A Cleaning Layer

Real transformations combine everything above. Here is the pattern you will write most often as a data engineer: read from a raw table, standardize each column, derive new ones, and keep raw and clean values side by side.

What you will learn

- Combine string, date, numeric and CASE logic in one SELECT
- Keep raw columns and add clean columns next to them
- Materialize the result as a table (CTAS) for the next layer

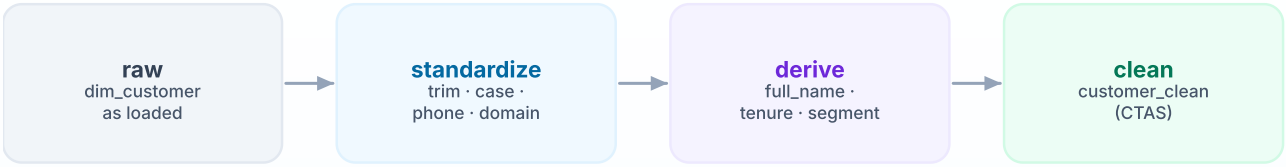


Figure 9. A transformation layer reads raw data and produces a trustworthy table; each layer has one clear job.

SQL — One SELECT, five kinds of transformation

```sql
SELECT customer_key,
       CONCAT(first_name, ' ', last_name)                    AS full_name,
       UPPER(LEFT(gender, 1))                                 AS gender_code,
       LOWER(TRIM(email))                                     AS email_clean,
       SUBSTRING_INDEX(email, '@', -1)                        AS email_domain,
       RIGHT(phone, 10)                                       AS phone_10,
       DATE_FORMAT(join_date, '%Y-%m')                        AS join_month,
       TIMESTAMPDIFF(YEAR, join_date, '2025-12-31')           AS tenure_years,
       CASE WHEN TIMESTAMPDIFF(YEAR, join_date, '2025-12-31') >= 4 THEN 'loyal'
            WHEN TIMESTAMPDIFF(YEAR, join_date, '2025-12-31') >= 2 THEN 'established'
            ELSE 'new' END                                    AS segment
FROM   dim_customer
ORDER  BY customer_key
LIMIT  5;
```

RESULT — 5 ROWS

customer_key	full_name	gender_code	email_clean	email_domain	phone_10	join_month	tenure_years	segment
1	Antonio Kelly	M	richard16@example…	example.net	9904005277	2021-09	4	loyal
2	Todd Lucas	F	danielsnyder@exam…	example.org	2907800627	2023-08	2	established
3	Matthew Lee	M	joseph40@example.…	example.net	6505299143	2022-03	3	established
4	Gregory Young	F	juan09@example.net	example.net	6936555456	2022-08	3	established
5	Nathan Smith	F	wroberts@example.…	example.net	3807169186	2022-04	3	established

SQL — CTAS the clean layer, then summarize it

```sql
CREATE TABLE customer_clean AS
SELECT customer_key, CONCAT(first_name, ' ', last_name) AS full_name,
       LOWER(TRIM(email)) AS email_clean, RIGHT(phone, 10) AS phone_10,
       CASE WHEN TIMESTAMPDIFF(YEAR, join_date, '2025-12-31') >= 4 THEN 'loyal'
            WHEN TIMESTAMPDIFF(YEAR, join_date, '2025-12-31') >= 2 THEN 'established'
            ELSE 'new' END AS segment
FROM   sales.dim_customer;
SELECT segment, COUNT(*) AS customers FROM customer_clean GROUP BY segment ORDER BY customers DESC;
```

RESULT — 3 ROWS

segment	customers
established	207
new	181
loyal	112

DATA-ENGINEERING LENS

Idempotent and testable. This transformation reads only from raw and writes only to the clean table, so re-running it gives the same result. Tools like dbt formalize exactly this: one SELECT per model, plus tests (`not_null` , `unique` , `accepted_values`) that run the profiling queries from Phase 3 automatically.

5.7 WRAP-UP

Phase 5 Recap & Practice

A function reference for the whole phase, plus a lab where you build a cleaning query yourself.

Cheat sheet

Area	Functions to remember
Numeric	`+ - * /` · `DIV` · `%` / `MOD` · `ROUND` · `FLOOR` · `CEIL` · `TRUNCATE` · `ABS` · `POWER` · `SQRT` · `GREATEST/LEAST`
String	`CONCAT` · `CONCAT_WS` · `LENGTH` · `UPPER/LOWER` · `TRIM` · `SUBSTRING` · `LEFT/RIGHT` · `INSTR` · `REPLACE` · `SUBSTRING_INDEX` · `REGEXP_REPLACE`
Date	`NOW` · `CURDATE` · `UTC_TIMESTAMP` · `YEAR/MONTH/DAY` · `DAYNAME` · `DATEDIFF` · `TIMESTAMPDIFF` · `DATE_ADD` · `DATE_FORMAT` · `STR_TO_DATE` · `LAST_DAY`
Casting	`CAST(x AS SIGNED|DECIMAL(p,s)|CHAR|DATE|DATETIME)` · `CONVERT(x, type)`
Conditional	`CASE WHEN … THEN … ELSE … END` · `IF(c,a,b)` · conditional aggregation `SUM(CASE …)`
NULL	`COALESCE` · `IFNULL` · `NULLIF` · `IS NULL` · `CONCAT_WS` skips NULLs
Rules of thumb	Cast explicitly · keep raw + clean columns · leave indexed columns untouched in WHERE · store UTC

Check yourself

Q1 What is the difference between `CONCAT` and `CONCAT_WS` with NULL inputs?

CONCAT returns NULL if any argument is NULL; CONCAT_WS skips NULL arguments.

Q2 `DIV` vs `/` ?

`/` gives a decimal quotient (7/2 = 3.5); `DIV` gives the integer part (3).

Q3 Why is `WHERE YEAR(order_date) = 2024` slower than a date range?

The function on the column prevents index use; the range lets MySQL seek directly.

Q4 How do you avoid divide-by-zero errors?

`a / NULLIF(b, 0)` returns NULL instead of failing.

Q5 How do you pivot rows into columns in MySQL?

Conditional aggregation: `SUM(CASE WHEN col = 'x' THEN val END)` per output column, with GROUP BY on the row key.

Practice lab

1. For every product, show `product_name` in UPPER case and its price rounded to the nearest 10 (`ROUND(x, -1)`). Show 3 rows.
2. Return the 5 customers with the longest full names (first + last), with the length.
3. How many customers joined on a **weekend**? Use a date function, not `dim_date`.
4. Show monthly revenue for 2024 (`YYYY-MM` + total), ordered by month. (hint: join `dim_date`)
5. Label each store `region` sale as `'big'` when `total_amount >= 5000`, else `'small'`, and count each label per region.

Answer key

SQL 1

```sql
SELECT UPPER(product_name) AS name_upper, unit_price, ROUND(unit_price, -1) AS price_10 FROM
dim_product ORDER BY product_key LIMIT 3;
```

RESULT 3 ROWS

name_upper	unit_price	price_10
FIGURE METHOD	897.35	900
PRESSURE THAT	920.58	920
HUGE CHANGE	553.56	550

SQL 2

```sql
SELECT CONCAT(first_name, ' ', last_name) AS full_name,
       CHAR_LENGTH(CONCAT(first_name, ' ', last_name)) AS len
FROM   dim_customer
ORDER  BY len DESC, full_name
LIMIT  5;
```

RESULT 5 ROWS

full_name	len
Christopher Mitchell	20
Christopher Robinson	20
Christopher Thompson	20
Deborah Blankenship	19
Elizabeth Fernandez	19

SQL — 3 · WEEKDAY: Monday = 0, so Saturday = 5 and Sunday = 6

```sql
SELECT COUNT(*) AS weekend_joiners FROM dim_customer WHERE WEEKDAY(join_date) IN (5, 6);
```

RESULT — 1 ROW

weekend_joiners
135

SQL — 4

```sql
SELECT   DATE_FORMAT(d.date, '%Y-%m') AS ym, ROUND(SUM(f.total_amount)) AS revenue
FROM     fact_sales f JOIN dim_date d ON d.date_key = f.date_key
WHERE    d.year = 2024
GROUP BY ym ORDER BY ym;
```

RESULT — 12 ROWS

ym	revenue
2024-01	690291
2024-02	618678
2024-03	590172
2024-04	613941
2024-05	588144
2024-06	589776
2024-07	583645
2024-08	604382
2024-09	473587
2024-10	583660
2024-11	500549
2024-12	671206

SQL 5

```sql
SELECT   s.region,
         CASE WHEN f.total_amount >= 5000 THEN 'big' ELSE 'small' END AS size_label,
         COUNT(*) AS sales
FROM     fact_sales f JOIN dim_store s ON s.store_key = f.store_key
GROUP BY s.region, size_label
ORDER BY s.region, size_label;
```

RESULT 10 ROWS

region	size_label	sales
Central	big	232
Central	small	1070
East	big	165
East	small	737
North	big	137
North	small	653
South	big	189
South	small	796
West	big	194
West	small	827

COMING IN PHASE 6

Phase 6: Advanced Analytics. Window functions, ranking functions, subqueries and CTEs: the tools behind running totals, Top-N per group, and readable multi-step queries.